

Họ & tên: Dương Thiện Quý

Mssv: 3123410298

BÀI THỰC HÀNH SỐ 6

Bài 2:

Tiến trình con ghi một mảng vào SM_0, với SM[0] chứa số phần tử mảng. Tiến trình cha

thực hiện, tính tổng các phần tử của mảng và ghi vào cuối SM_1. Trình con trình nhận và xuất

dữ liệu ra màn hình (lưu ý sử dụng 2 SM).

Code:

```
OS_HomeWork > BT_Tuan4 > C bai2.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/ipc.h>
5  #include <sys/shm.h>
6  #include <sys/wait.h>
7  #include <time.h>
8
9  #define MAX_SIZE 100
10
11 // Thiết kế một Struct rõ ràng để quản lý trạng thái, tránh giá trị rác của RAM
12 typedef struct {
13     int n;
14     int data[MAX_SIZE];
15     int sum;
16     int status; // 0: Con đang tạo mảng, 1: Cha đang tính tổng, 2: Con đọc kết quả
17 } SharedBuffer;
18
19 int main() {
20     // 1. Tạo 2 key và xin cấp phát 2 vùng nhớ chia sẻ
21     key_t key0 = ftok("bai2.c", 65);
22     key_t key1 = ftok("bai2.c", 66);
23
24     int shmid0 = shmget(key0, sizeof(SharedBuffer), 0666 | IPC_CREAT);
25     int shmid1 = shmget(key1, sizeof(int), 0666 | IPC_CREAT);
26
27     if (shmid0 == -1 || shmid1 == -1) {
28         perror("shmget failed");
29         exit(1);
30     }
31
32     // Gắn vùng nhớ ở cha trước để thiết lập trạng thái ban đầu ổn định
33     SharedBuffer *shared_init = (SharedBuffer*) shmat(shmid0, NULL, 0);
34     shared_init->status = 0; // Đặt status = 0 để báo hiệu con chạy trước
35     shared_init->n = 0;
```

```

OS_HomeWork > BT_Tuan4 > C bai2.c > main()
32 // Chia vùng nhớ ở chương trình cha thành 2 tập con: con 1 và con 2
33 SharedBuffer *shared_init = (SharedBuffer*) shmat(shmid0, NULL, 0);
34 shared_init->status = 0; // Đặt status = 0 để báo hiệu con chạy trước
35 shared_init->n = 0;
36 shmdt(shared_init);
37
38 int *sm1_init = (int*) shmat(shmid1, NULL, 0);
39 sm1_init[0] = 0;
40 shmdt(sm1_init);
41
42 // 2. Tạo tiến trình con
43 pid_t pid = fork();
44
45 if (pid < 0) {
46     perror("fork failed");
47     exit(1);
48 }
49
50 if (pid == 0) { // === TIỀN TRÌNH CON ===
51     // Tự gán vùng nhớ vào không gian địa chỉ của con
52     SharedBuffer *shared = (SharedBuffer*) shmat(shmid0, NULL, 0);
53     int *sm1 = (int*) shmat(shmid1, NULL, 0);
54
55     srand(time(NULL) ^ getpid());
56     int count = rand() % 10 + 5; // Sinh mảng từ 5 đến 14 phần tử
57     shared->n = count;
58
59     printf("Tiến trình con: Tạo mảng ngẫu nhiên gồm %d phần tử!\n", count);
60     for(int i = 0; i < count; i++) {
61         shared->data[i] = rand() % 100;
62         printf("%d ", shared->data[i]);
63     }
64     printf("\n");
65
66     // Đổi trạng thái sang 1 để báo cho cha biết con đã ghi xong mảng

```

```

OS_HomeWork > BT_Tuan4 > C bai2.c > main()
64     printf("\n");
65
66     // Đổi trạng thái sang 1 để báo cho cha biết con đã ghi xong mảng
67     shared->status = 1;
68
69     // Chờ tiến trình cha tính toán xong (Đợi sm1[0] có kết quả khác 0)
70     while (sm1[0] == 0) {
71         usleep(10000); // Ngủ 10ms để nhường CPU cho cha tính
72     }
73
74     printf("Tiến trình con: Nhận được kết quả tổng từ cha = %d\n", sm1[0]);
75
76     // Tháo các vùng nhớ trước khi thoát
77     shmdt(shared);
78     shmdt(sm1);
79     exit(0);
80
81 } else { // === TIỀN TRÌNH CHA ===
82     SharedBuffer *shared = (SharedBuffer*) shmat(shmid0, NULL, 0);
83
84     // Đợi cho đến khi status chuyển sang 1 (Con ghi xong mảng)
85     while (shared->status != 1) {
86         usleep(10000);
87     }
88
89     int total_elements = shared->n;
90     int current_sum = 0;
91
92     // Tính tổng
93     for(int i = 0; i < total_elements; i++) {
94         current_sum += shared->data[i];
95     }
96     printf("Tiến trình cha: Đã tính xong tổng mảng = %d. Đang gửi sang SM_1...\n", cu
97
98     // Tháo SM_0 sau khi dùng xong

```

```

OS_HomeWork > BT_Tuan4 > C bai2.c > main()
88
89     int total_elements = shared->n;
90     int current_sum = 0;
91
92     // Tính tổng
93     for(int i = 0; i < total_elements; i++) {
94         current_sum += shared->data[i];
95     }
96     printf("Tiến trình cha: Đã tính xong tổng mảng = %d. Đang gửi sang SM_1...\n", cu
97
98     // Tháo SM_0 sau khi dùng xong
99     shmdt(shared);
100
101     // Gắn vùng nhớ SM_1 để ghi kết quả cho con nhận
102     int *sm1 = (int*) shmat(shmid1, NULL, 0);
103     sm1[0] = current_sum;
104     shmdt(sm1); // Tháo SM_1
105
106     // Đợi tiến trình con in kết quả và kết thúc hoàn toàn
107     wait(NULL);
108
109     // 6. Hủy phân vùng Shared Memory khỏi hệ thống RAM
110     shmctl(shmid0, IPC_RMID, NULL);
111     shmctl(shmid1, IPC_RMID, NULL);
112     printf("Tiến trình cha: Đã hủy cả 2 vùng nhớ dùng chung và kết thúc hệ thống.\n");
113
114
115     return 0;
116 }

```

Kết quả:

```

❖ quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan4$ ./bai2.out
Tiến trình con: Tạo mảng ngẫu nhiên gồm 11 phần tử:
91 19 14 91 80 83 64 4 40 78 15
Tiến trình con: Nhận được kết quả tổng từ cha = 579
Tiến trình cha: Đã hủy cả 2 vùng nhớ dùng chung và kết thúc.
❖ quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan4$

```

Bài 3:

Viết 1 chương trình, xây dựng 3 tiến trình P0, P1, P2. Tạo ra 1 file “data” file này chứa các

dãy số nguyên, dãy số này có thể có số phần tử khác nhau.

- P0 đọc các dãy số, sau đó chép lên share memory.
- P1 truy cập vào share memory, sắp xếp các dãy số theo thứ tự tăng dần, và tính tổng cho dãy số đó.
- Sau khi P1 làm xong, P2 sẽ truy cập vào share memory, sắp xếp các dãy số này sao cho dãy số nào có tổng nhỏ nhất, đặt ở trên, dãy nào có tổng lớn hơn đặt ở dưới.
- Sau khi làm xong, P0 sẽ chép vào file “result”.

Code:

```
OS_HomeWork > BT_Tuan4 > C bai3.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/ipc.h>
5  #include <sys/shm.h>
6  #include <sys/wait.h>
7  #include <string.h>
8
9  #define MAX_ARRAYS 10
10 #define MAX_ELEM 50
11
12 // Cấu trúc struct trong Shared Memory để lưu trữ dãy số
13 typedef struct {
14     int count;           // Số lượng phần tử trong dãy
15     int data[MAX_ELEM]; // Mảng chứa các phần tử
16     int sum;             // Biến lưu tổng của dãy
17 } Sequence;
18
19 // Cấu trúc dùng chung cho cả bộ nhớ chia sẻ
20 typedef struct {
21     int num_sequences;    // Tổng số dãy số
22     Sequence seqs[MAX_ARRAYS]; // Mảng các dãy số
23     // Biến trạng thái để đồng bộ (Pipeline: P0 -> P1 -> P2 -> P0)
24     // 0: P0 đang đọc file; 1: P1 đang xử lý; 2: P2 đang xử lý; 3: P0 ghi file
25     int state;
26 } SharedData;
27
28 // Hàm sắp xếp mảng tăng dần
29 void sort_asc(int arr[], int n) {
30     for (int i = 0; i < n - 1; i++) {
31         for (int j = i + 1; j < n; j++) {
32             if (arr[i] > arr[j]) {
33                 int temp = arr[i];
34                 arr[i] = arr[j];
35                 arr[j] = temp;
36             }
37         }
38     }
39 }
```


OS_HomeWork > BT_Tuan4 > C bai3.c > ...

```
27
28 // Hàm sắp xếp mảng tăng dần
29 void sort_asc(int arr[], int n) {
30     for (int i = 0; i < n - 1; i++) {
31         for (int j = i + 1; j < n; j++) {
32             if (arr[i] > arr[j]) {
33                 int temp = arr[i];
34                 arr[i] = arr[j];
35                 arr[j] = temp;
36             }
37         }
38     }
39 }
40
41 // Hàm sắp xếp các dãy số theo tổng (tổng nhỏ ở trên, lớn ở dưới)
42 void sort_seqs(Sequence seqs[], int n) {
43     for (int i = 0; i < n - 1; i++) {
44         for (int j = i + 1; j < n; j++) {
45             if (seqs[i].sum > seqs[j].sum) {
46                 Sequence temp = seqs[i];
47                 seqs[i] = seqs[j];
48                 seqs[j] = temp;
49             }
50         }
51     }
52 }
53
54 int main() {
55     // Tạo key dùng chung
56     key_t key = ftok("bai3.c", 67);
57
58     // Khởi tạo vùng nhớ chia sẻ cho SharedData
59     int shmid = shmget(key, sizeof(SharedData), 0666 | IPC_CREAT);
60     if (shmid == -1) {
61         perror("lỗi shmget");
62     }
63 }
```

OS_HomeWork > BT_Tuan4 > C bai3.c > ...

```
54 int main() {
60     if (shmid == -1) {
61         perror("lỗi shmget");
62     }
63
64     // KHÔNG gọi shmat() trước fork() để tránh kế thừa thụ động
65
66     pid_t pid1, pid2;
67
68     // Tạo tiến trình P1
69     pid1 = fork();
70     if (pid1 == 0) { // Đây là P1
71         // Gắn vùng nhớ vào không gian địa chỉ của P1
72         SharedData *shared = (SharedData*) shmat(shmid, NULL, 0);
73
74         // Chờ P0 hoàn thành việc chép dữ liệu lên SM
75         while (shared->state != 1) {
76             usleep(10000); // Đợi 10ms thay vì 1s
77         }
78
79         printf("P1: Đã nhận tín hiệu. Bắt đầu sắp xếp tăng dần và tính tổng từng dãy...\n");
80         for (int i = 0; i < shared->num_sequences; i++) {
81             // Sắp xếp tăng dần
82             sort_asc(shared->seqs[i].data, shared->seqs[i].count);
83
84             // Tính tổng
85             int sum = 0;
86             for (int j = 0; j < shared->seqs[i].count; j++) {
87                 sum += shared->seqs[i].data[j];
88             }
89             shared->seqs[i].sum = sum; // Ghi kết quả lên SM
90         }
91     }
92
93     // Chuyển quyền cho P2
94     shared->state = 2;
95     shmdt(shared); // Tạm tháo vùng nhớ
96 }
```

OS_HomeWork > BT_Tuan4 > C bai3.c > ...

```
54 int main() {
71     if (pid1 == 0) { // Đây là P1
93         // Chuyển quyền cho P2
94         shared->state = 2;
95         shmdt(shared); // Tạm tháo vùng nhớ
96         exit(0);
97     }
98
99     // Tạo tiến trình P2
100    pid2 = fork();
101    if (pid2 == 0) { // Đây là P2
102        // Gắn vùng nhớ vào không gian địa chỉ của P2
103        SharedData *shared = (SharedData*) shmat(shmid, NULL, 0);
104
105        // Chờ P1 xử lý xong
106        while (shared->state != 2) {
107            usleep(10000); // Đợi 10ms thay vì 1s
108        }
109
110        printf("P2: Đã nhận tín hiệu. Bắt đầu sắp xếp các dãy theo tổng...\n");
111        sort_seqs(shared->seqs, shared->num_sequences);
112
113        // Chuyển quyền lại cho P0
114        shared->state = 3;
115        shmdt(shared); // Tạm tháo vùng nhớ
116        exit(0);
117    }
118
119    // Tiến trình P0
120    // Gắn vùng nhớ vào không gian địa chỉ của P0
121    SharedData *shared = (SharedData*) shmat(shmid, NULL, 0);
122
123    // Khởi tạo các giá trị ban đầu cho bộ nhớ chia sẻ
124    shared->state = 0;
```

OS_HomeWork > BT_Tuan4 > C bai3.c > ...

```
54 int main() {
123    // Khởi tạo các giá trị ban đầu cho bộ nhớ chia sẻ
124    shared->state = 0;
125    shared->num_sequences = 0;
126
127    printf("P0: Đang đọc file data.txt và chép lên Shared Memory...\n");
128    FILE *fin = fopen("data.txt", "r");
129    if (!fin) {
130        perror("Lỗi mở file data.txt");
131        // Hủy vùng nhớ và kết thúc nếu không có file
132        shmctl(shmid, IPC_RMID, NULL);
133        shmdt(shared);
134        exit(1);
135    }
136
137    char line[256];
138    int seq_idx = 0;
139    while (fgets(line, sizeof(line), fin) && seq_idx < MAX_ARRAYS) {
140        char *token = strtok(line, " \t\n");
141        int count = 0;
142        while (token != NULL && count < MAX_ELEM) {
143            shared->seqs[seq_idx].data[count++] = atoi(token);
144            token = strtok(NULL, " \t\n");
145        }
146        if (count > 0) {
147            shared->seqs[seq_idx].count = count;
148            seq_idx++;
149        }
150    }
151    fclose(fin);
152    shared->num_sequences = seq_idx;
153
154    // Cho phép P1 bắt đầu
155    shared->state = 1;
```

```

OS_HomeWork > BT_Tuan4 > C bai3.c > ...
54  int main() {
152      shared->num_sequences = seq_idx;
153
154      // Cho phép P1 bắt đầu
155      shared->state = 1;
156
157      // P0: Chờ P2 hoàn thành (state == 3)
158      while (shared->state != 3) {
159          usleep(10000); // Đợi 10ms
160      }
161
162      printf("P0: Nhận dữ liệu cuối cùng từ P2. Ghi ra result.txt...\n");
163      FILE *fout = fopen("result.txt", "w");
164      if (fout) {
165          for (int i = 0; i < shared->num_sequences; i++) {
166              fprintf(fout, "Sum: %d | Data: ", shared->seqs[i].sum);
167              for (int j = 0; j < shared->seqs[i].count; j++) {
168                  fprintf(fout, "%d ", shared->seqs[i].data[j]);
169              }
170              fprintf(fout, "\n");
171          }
172          fclose(fout);
173      }
174
175      // Chờ 2 tiến trình con kết thúc hoàn toàn
176      wait(NULL);
177      wait(NULL);
178
179      // Hủy vùng nhớ chia sẻ
180      shmctl(shmid, IPC_RMID, NULL);
181      shmdt(shared);
182      printf("P0: Đã ghi xong result.txt, dọn dẹp bộ nhớ và kết thúc!\n");
183
184      return 0;
185  }

```

File data:

```

OS_HomeWork > BT_Tuan4 > ≡ data.txt
1  12 5 8 1 9 4
2  3 15 7
3  20 11 34 2 9 6 1
4  45 10 23 2
5

```

Kết quả:

OS_HomeWork > BT_Tuan4 >  result.txt

```
1 Sum: 25 | Data: 3 7 15
2 Sum: 39 | Data: 1 4 5 8 9 12
3 Sum: 80 | Data: 2 10 23 45
4 Sum: 83 | Data: 1 2 6 9 11 20 34
5
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan4$ ./bai3.out
P0: Đang đọc file data.txt và chép lên Shared Memory...
P1: Đã nhận tín hiệu. Bắt đầu sắp xếp tăng dần và tính tổng từng dãy...
P2: Đã nhận tín hiệu. Bắt đầu sắp xếp các dãy theo tổng...
P0: Nhận dữ liệu cuối cùng từ P2. Ghi ra result.txt...
P0: Đã ghi xong result.txt, dọn dẹp bộ nhớ và kết thúc!
❖ quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan4$
```